Table of Contents

1. Product Charter 2

2. Product Specification 3

2.1 Information Architecture . . . 3

2.2 User Flows . . . 3

2.3 Rating Card — the Nine Categorical States . . . 3

2.4 Alert Rules . . . 4

2.5 Copy Library . . . 5

2.6 Free / Pro Boundary Without Selling Picks . . . 5

3. Database Schema 5

4. API Specification 6

5. Component Specifications 7

5.1 Rating Card . . . 7

5.2 Luck Meter . . . 8

6. Statistical Rules and the Preregistered Gate 9

7. Test Suite Report 10

8. Launch Checklist 11

Appendix A. Prohibited Vocabulary Ruleset 12

Appendix B. Reference Deployment Notes 12

1. Product Charter

PropStein is an evidence-first sports-prop research and record-keeping product. Its purpose is to help a user understand **price, uncertainty, provenance, and their own historical record**. It is explicitly not a betting recommendation product: it contains no picks, no sizing guidance, no urgency language, and no actionable-signal claims before preregistered evidence exists. The product wins by being the one that refuses to claim an advantage before the evidence exists.

The core loop is: upload or enter a slip, confirm the parsed selections, compare the price and line to available market and model distributions, save an immutable ledger entry, capture a closing reference when available, settle and grade, and finally show a calibrated record with honest uncertainty. Every surface in the build implements this loop and nothing beyond it.

The ten non-negotiable rules, implemented and tested:

- **No picks.** The system never outputs or implies an instruction to take or decline a price. The nine allowed categoricals are enumerated in Chapter 2.
- **No unproven advantage claims.** The words the charter prohibits are absent from every surface until a sport has passed its preregistered commercialization gate; in this build they are absent entirely.
- **No sizing advice.** Wager amounts are optional user-entered record fields, never used in assessments or advice.
- **No urgency.** No pressure language anywhere; the copy linter enforces this mechanically.
- **No fabricated confidence.** Every model output carries version, uncertainty interval, data freshness, provenance, and coverage status.
- **No silent exclusions.** Missing, stale, conflicting, void, canceled, push, and DNP records are retained with explicit reason codes.
- **No cross-sport pooling.** NFL evidence never authorizes NBA or UFC claims; the gate engine raises an error on pooled input.
- **No retroactive mutation.** Ledger records are append-only; corrections create revisions with reasons and integrity hashes.
- **No LLM in the decision path.** The vision model performs slip-field extraction only; every probability, rating, and gate decision is computed by deterministic code.
- **Fail closed.** Missing provenance, freshness, entitlement, coverage, or closing-reference quality produces an explicit unavailable state, never an invented answer.

> *This is not a betting recommendation. PropStein records prices, uncertainty, provenance, and the user's own record — nothing else.*

2. Product Specification

2.1 Information Architecture

Surface	Purpose
Slip	Upload / paste / manual entry, extraction confirmation, price assessment, save to ledger.
Ledger	Append-only record list; entry detail with Evidence DNA, revision history, closing reference, settlement grade, integrity check.
Record	Luck Meter: descriptive statistics, exclusion histogram, verdict ladder, calibration, slices.
Alerts	Opt-in factual tracked-entry updates with snapshot time, source, and ledger linkage.
Gate Monitor	Preregistered gate state per sport: parameters, progress, attempt history, compliance approval, commercial flag (default deny).
Docs	In-product specification, copy-compliance report, launch checklist.

Table 2-1. Terminal surfaces.

2.2 User Flows

Flow A — screenshot. The user uploads one or more slip screenshots. A vision model extracts sportsbook, sport, league, event, player, market, selection, line, and American odds, with per-field confidence. The confirmation screen shows every field as editable; fields below the 0.75 confidence threshold are highlighted for manual review. Nothing is analyzed until the user checks the explicit confirmation box and presses the confirm button — the API itself rejects unconfirmed submissions. Screenshots are stored only when the user opts in, and deletion is available. The system never requests or retains sportsbook credentials, cookies, balances, or account data, and never submits, prepares, or automates wagers.

Flow B — pasted text. The user pastes slip text; a deterministic parser extracts the same field set with confidence scores, then the same confirmation and assessment path runs.

Flow C — manual entry. The user types the fields directly; the record is marked manual_entry in the ledger and follows the same confirmation gate.

Flow D — grading loop. A pending entry receives a closing reference from a provider independent of the model input feed (quality states: fresh, stale, missing, conflicting). Settlement then runs (settled, push, void, canceled, DNP — each with a reason code). The Luck Meter consumes only evaluable entries; excluded entries are retained and reported in the reason-code histogram, never counted as losses or zeros.

2.3 Rating Card — the Nine Categorical States

State	Trigger	Display copy (exact)
PRICE ABOVE MARKET HURDLE	Fresh licensed comparison; price delta below −0.25 pp	Entered price is X.X percentage points better than the comparison snapshot
PRICE AT MARKET HURDLE	Delta within ±0.25 pp	Entered price is at the comparison snapshot within the preregistered band
PRICE BELOW MARKET HURDLE	Delta above +0.25 pp	Entered price is X.X percentage points worse than the comparison snapshot
INSUFFICIENT MARKET DATA	No licensed comparison available	Break-even probability still shown because it derives from the entered price alone. This is not an instruction to take or decline the price.
STALE QUOTE	Comparison age over 300 seconds	Comparison snapshot is N seconds old (threshold 300 seconds). The comparison is retained but not used.
NO MODEL COVERAGE	Model surface requested; no calibrated coverage	No calibrated model coverage exists for this market. This is not an instruction to take or decline the price.
UNVERIFIED CONTEXT	Spread, total, or weather observed-at missing or after the pricing cutoff	Context inputs are not verified as known before the pricing cutoff. The assessment fails closed.
MODEL UNAVAILABLE	Model surface outage	The model registry lists this market, but the model surface is unavailable. The assessment fails closed.
SETTLED	Settlement recorded	Settlement recorded with status S and result R.

Table 2-2. The complete categorical output space. No other categorical exists in the product.

Error and empty states follow the same discipline: loading, unconfirmed extraction, entitlement failure, kill switch active, and service unavailable each render an explicit fail-closed card; the unavailable card states that nothing was invented to fill the gap. Accessibility: the categorical chip is role=status with aria-live so screen readers announce the assessment; every section is labelled; the persistent banner and footer carry the disclaimer.

2.4 Alert Rules

- Opt-in by alert type; daily cap of 3 push notifications per user; non-urgent updates are batched.
- Every alert displays snapshot time and data source and links to the immutable ledger entry.
- If freshness cannot be verified, a STALE QUOTE state is sent or the alert is suppressed.
- Allowed messages are factual and past-tense: saved quote changed from −115 to −135; saved quote is now 9 minutes old; player status changed from Questionable to Out; market is suspended; closing reference is now available; saved entry has settled; saved entry is unevaluable because the closing reference is stale.
- Instructions to enter, exit, resize, or time anything are structurally absent: the message templates are machine-checked by the copy linter, and a template failing the lint is rejected at delivery time (LINT_REJECTED).

2.5 Copy Library

Required vocabulary, used verbatim across the product: price assessment, break-even probability, market comparison, snapshot time, data freshness, model coverage, uncertainty interval, no calibrated coverage, insufficient evidence, descriptive only, recorded for review, closing reference unavailable, this is not a betting recommendation. The runtime copy registry is scanned by the compliance linter; the Docs surface shows the live report (167 tests include a full-surface copy scan with positive controls). Appendix A enumerates the prohibited vocabulary and the reasoning behind each safe-context allowance.

2.6 Free / Pro Boundary Without Selling Picks

Tier	Included
Free	Slip upload and manual entry; hurdle-router price assessments; the immutable ledger; the Luck Meter; standard alerts.
Pro	Additional licensed comparison feeds by sport; closing-reference capture for additional leagues; longer retention windows; data export.
Neither tier	Picks, signal strengths, sizing guidance, or actionable-signal language. Entitlement gates data surfaces only; verification is server-side only.

Table 2-3. Plan boundary: data surfaces only, never advice.

3. Database Schema

The reference deployment uses SQLite through Prisma; production uses PostgreSQL with identical models. Redis backs the jti revocation set, the sliding-window rate counters, and the kill-switch cache in production; the reference build implements the same interfaces in-process. Object storage holds screenshots and raw provider captures.

Model	Fields and semantics
User	Pseudonymous id, plan, retention defaults. No credentials or sportsbook account linkage anywhere.
SessionToken	jti, kind (web / extension), scopes, expiry, revocation timestamp.
SlipUpload	Source type, OCR provider and confidence, extracted fields JSON, retention flag, deletion and confirmation timestamps, raw capture reference.
Assessment	Canonical HurdleInput JSON, snapshot hash, result JSON (categorical, model block, provenance), scenario, links to slip upload and ledger entry.
LedgerEntry	Immutable core: ids, sport, event, game cluster, player, market, selection, line, two-way entered quote, implied and break-even probabilities, book, captured-at, snapshot hash, model version / coverage / probability / uncertainty, price assessment, provenance JSON, entry hash, duplicate and correlation flags.

Model	Fields and semantics
LedgerRevision	Append-only: revision number, reason code and text, actor, changed fields, prior hash, new hash, timestamp.
MarketSnapshot	Comparison quotes: provider, book, two-way odds, captured time, raw reference, snapshot hash.
ClosingReference	Provider, captured time, two-way close, implied and fair probabilities, quality (fresh / stale / missing / conflicting), second source, independence flag, integrity hash.
Settlement	Status (pending / settled / void / canceled / push / dnp), result, reason code, actual statistic, source, settled timestamp.
AlertEvent / AlertPreference	Factual alerts with snapshot time, source, ledger linkage, delivery state, suppression reason; per-type opt-in.
Entitlement	Surface, scope, status, source, expiry — server-side verification only.
AuditEvent	Actor, action, target, request id, metadata — structured JSON logging with request id, pseudonym, model version, provenance ids.
GateSignal / GateAttempt / ComplianceApproval	Per-sport gate evidence: signals with cluster ids and reason codes, attempt results with seed and bounds, compliance approval records.
SystemFlag	Kill switches (extension, market intelligence) and model outage flags.

Table 3-1. Models. No update path exists for LedgerEntry anywhere in the API.

Append-only integrity. The entry hash is the SHA-256 digest of the canonical JSON of the immutable core. The genesis revision binds the entry hash. Every correction appends a revision whose new hash chains from the prior hash; forbidden fields (ledgerId, entryHash, createdAtUtc, snapshotHash) are rejected. Verification recomputes the chain and reports mismatches as integrity failures, which the ledger detail surface displays. Settlements and closing references are child records, never mutations.

4. API Specification

All endpoints return JSON envelopes: { ok: true, ... } or { ok: false, code, detail }. Fail-closed errors are explicit states. Rate limits are sliding-window per IP and route (12/min on upload, 20/min on confirm, 30/min on ledger writes). There is deliberately no PUT or PATCH route for ledger entries, and no bet placement, preparation, or submission route anywhere in the surface — the test suite asserts both properties against the route table.

Method	Path	Purpose
POST	`/api/slips/upload`	Screenshot (multipart) or pasted text (JSON) to extraction with confidence; never analyzes.
POST	`/api/slips/confirm`	Explicit confirmation gate; runs the assessment pipeline; persists an Assessment. Rejects confirm !== true with CONFIRMATION_REQUIRED.

Method	Path	Purpose
GET	`/api/assessments/{id}`	Assessment bundle: rating card inputs, model block, provenance.
POST	`/api/ledger/entries`	Save the immutable ledger entry from a confirmed assessment; 409 on double-save; 503 fail-closed on model outage.
GET	`/api/ledger`	List entries; filters by sport and settlement status.
GET	`/api/ledger/{id}`	Full detail: core, provenance, revisions, closing reference, settlement, grade, hash-chain integrity.
POST	`/api/ledger/{id}/close`	Capture the closing reference with quality states and unevaluable reasons.
POST	`/api/ledger/{id}/settle`	Settle and grade: win / loss / push / void / canceled / DNP with reason codes.
POST	`/api/ledger/{id}/correction`	Append a revision; requires a substantive reason; rejects empty changes and immutable fields.
GET	`/api/records/luck-meter`	Luck Meter report: verdict ladder, exclusions, intervals, slices, calibration.
POST / GET	`/api/alerts/preferences`	Opt-in by alert type; current preferences.
GET	`/api/alerts`	Alert feed with snapshot time, source, ledger linkage.
POST	`/api/alerts/evaluate`	Run the alert evaluation worker (preferences, mute, daily cap, freshness gate).
POST	`/api/extension/session`	Issue a read-only extension session: scopes fixed to market.observe.readonly + ledger.append; kill-switch refusal is explicit (503).
POST	`/api/extension/session/revoke`	Revoke by token or jti; revocation takes effect immediately.
GET	`/api/extension/health`	Kill-switch state, adapter versions, read-only posture.
GET	`/api/gates`	Gate monitor view per sport.
POST	`/api/gates/recompute`	Deterministic recompute from stored signals with the preregistered seed.
GET	`/api/copy-compliance`	Live copy lint report over the product copy registry.
GET	`/api/me`	Session pseudonym and plan.

Table 4-1. Complete API surface.

5. Component Specifications

5.1 Rating Card

The Rating Card renders a price assessment with required fields on two panels and a provenance strip. Panel one: entered price with implied and break-even probability, market comparison (provider, two-way quote,

implied probability), price difference in percentage points with the preregistered band, snapshot time in ET, and data freshness (age, threshold, state). Panel two: model coverage — name, version, probability and 95% uncertainty interval only when calibrated, calibration status and corpus, data freshness, known limitations (collapsible); uncalibrated coverage shows the model identity and withholds all numbers; no coverage shows "No calibrated coverage". The provenance strip lists every input with observed-at versus the pricing cutoff (filled marker = known before cutoff; triangle = required but unverified) and the snapshot hash. The footer carries the deterministic detail line and the fail-closed note.

State coverage: all nine categoricals plus loading, unconfirmed, entitlement failure, kill switch, and service unavailable — the latter three render the explicit fail-closed card. Accessibility: role=status, aria-live on the categorical, labelled sections, sr-only context line, full keyboard focus, and a persistent disclaimer.

5.2 Luck Meter

The Luck Meter reports the record honestly. Headline statistics: evaluable count; excluded count with the reason-code histogram (a bar chart, nothing silently dropped); mean price CLV; shrunk CLV estimate (N / (N + 100) shrinkage toward zero); the 95% game-clustered bootstrap interval (10,000 replicates, recorded seed); the share of entries beating close; and calibration (Brier score and decile table) where calibrated model probabilities exist. Slices by sport, market, odds band, model version, and entry source flag groups below the 30-entry minimum sample with an amber warning.

Verdict	Display rule (verbatim)
NO RECORD	No graded entries yet.
INSUFFICIENT SAMPLE	Not enough evaluable entries to draw a conclusion.
DESCRIPTIVE ONLY	This is descriptive. It does not establish an advantage over the market.
INCONCLUSIVE	The interval includes zero. Do not interpret this as an advantage over the market.
POSITIVE BUT UNPROVEN	Positive descriptive CLV. The preregistered confirmation gate has not passed.
GATE PASSED	Shown only after the sport-specific preregistered gate passes and a compliance approval record exists.

Table 5-1. The verdict ladder.

CLV definition: de-vigged closing fair probability minus de-vigged entry fair probability, in percentage points, using two-way proportional de-vig at both entry and close; positive means the recorded price was superior to the closing reference. Model-versus-close probability difference is a diagnostic only and is never presented as CLV. Forbidden-claim linting: the copy registry contains no claim strings, and the component file is scanned by the copy test with positive controls proving the linter catches each prohibited pattern.

6. Statistical Rules and the Preregistered Gate

The governing rule: a commercial actionable-signal product may launch for a sport only after that sport's gate passes — 600 evaluable qualifying signals, observed mean price CLV of at least +1.0 percentage point, and a game-clustered bootstrap lower bound above zero. Cross-sport pooling is forbidden. The preregistered constants are fixed in code; changing any of them is a protocol change requiring re-registration.

Parameter	Value	Enforcement
Confirmatory N	600 independently evaluable qualifying signals per sport	evaluateGateAttempt; accumulation states below N
Interim look	N = 300, descriptive only, never a verdict	INTERIM verdict carries the descriptive-only note
Effect threshold	Mean price CLV at least +1.0 pp	PASS requires mean >= threshold
One-sided alpha	0.0125 per attempt	Bootstrap lower bound is the 1.25th percentile of the mean
Bootstrap	Game-clustered, resampling clusters with replacement	Within-cluster dependence preserved by construction
Replicates	10,000	Fixed
Seed	20261024	Fixed; the Gate Monitor recompute control proves determinism
Attempts	Maximum two prospective attempts per sport	ProtocolError beyond the second
Cluster floor	30 distinct game clusters	INCONCLUSIVE below the floor
Model versions	A version change restarts confirmatory accumulation	Prior signals marked PRE_REGISTRATION
Commercial launch	Recorded gate PASS and explicit compliance approval; default deny	sportCommercialState
Retirement	A failed second attempt retires the sport's commercial actionable-signal flag	Retired state is permanent; compliance approval cannot override it

Table 6-1. Preregistered gate parameters and their enforcement.

Sign-convention note (preregistered interpretation). The governing formula sheet writes CLV(i) = p_fair(entry) − p_fair(close); read literally, that sign makes "beating the close" negative, which contradicts the sheet's own gate semantics (mean CLV at least +1.0 pp, share of entries beating close, positive descriptive CLV). The implemented convention is therefore the standard one — CLV(i) = p_fair(close) − p_fair(entry) — so that positive CLV means the entry price was superior to the closing reference. The convention is documented here and in the grading module, and the test suite pins it with exact fixtures.

Evidence handling: duplicates (identical deduplication keys) are excluded with reason DUPLICATE; correlated siblings (same game cluster and player) are reduced to the earliest per correlation group with reason CORRELATED while game cluster ids on retained rows are untouched, so the cluster bootstrap still sees the dependence; pushes, voids, cancellations, and DNPs are excluded with their reason codes; missing, stale, conflicting, or non-independent closing references make signals unevaluable — never zeros. Two independent closing sources must agree within 1.5 percentage points of implied probability.

7. Test Suite Report

The suite runs 167 tests across seven files (vitest, deterministic — no network, no clock dependence beyond fixed ISO timestamps). All pass. The nine required proof obligations map as follows:

#	Required proof	Test file and assertions
1	Push, void, DNP, cancellation excluded with reason codes	odds-grading.test.ts — gradeEntry returns evaluable=false with reasonCode PUSH / VOID / CANCELED / DNP and null CLV
2	Missing or stale closing prices become unevaluable, not zero	odds-grading.test.ts — MISSING_CLOSE and STALE_CLOSE return null CLV with reason codes
3	Duplicate and correlated signals deduplicated without destroying cluster dependence	gate.test.ts + odds-grading.test.ts — DUPLICATE and CORRELATED exclusions; retained rows keep cluster ids; bootstrap spread test shows clustering changes the replicate distribution
4	Source disagreement beyond tolerance becomes unevaluable	odds-grading.test.ts — closingQualityFromSources boundary at 1.5 pp; CONFLICTING_CLOSE grade; simulated conflicting fixture
5	Post-registration mutation is rejected	ledger.test.ts — APPEND_ONLY_VIOLATION guard throws; forbidden fields rejected; reason required; hash chain and tamper detection verified
6	Model-version changes stratify or invalidate confirmatory sets	gate.test.ts — mixed-version set marks prior signals PRE_REGISTRATION and restarts accumulation
7	Passing and failing gate scenarios produce exact verdicts	gate.test.ts — PASS (mean above threshold, bound above zero), FAIL (weak mean; strong mean with noisy bound), IN_PROGRESS below 300, INTERIM at 300–599
8	Insufficient cluster representation produces an inconclusive verdict	gate.test.ts — 600 signals in 12 clusters returns INCONCLUSIVE with the protocol note
9	A failed second attempt retires the commercial flag	gate.test.ts — sportCommercialState retires with two FAILs even when compliance is granted; PASS without compliance still denies

Table 7-1. Required proofs and their tests.

- Unit tests: implied probability and break-even (spec examples pinned: −115 implies 53.49%), proportional de-vig sums to one, price delta matches the specification example (−115 vs −112 is 0.7 pp worse), odds bands, staleness thresholds, unknown-timestamp handling.
- Contract tests: ledger hash chains verify; tampering with any core field breaks verification; corrections chain correctly; canonical hashing is key-order independent.

- Copy tests: positive controls prove the linter catches 30 offender strings (picks, sizing, urgency, claim language); the full product surface scan (copy registry, all component string literals and rendered JSX text, domain strings that render, in-app docs) returns zero findings; the nine allowed categoricals are clean.
- Extension safety: scopes fixed to read-only observation plus ledger append; forbidden scopes not issuable; HMAC verification rejects tampered and expired tokens; revocation is immediate; the kill switch refuses issuance; the route table contains no wagering paths and no PUT/PATCH/DELETE handlers on ledger routes.
- Luck Meter ladder: all six verdict states with the display rules verbatim; exclusions never dilute the mean; below-minimum slices flagged.

8. Launch Checklist

Review area	Status	Notes
Security	Complete in reference build	Server-side entitlement verification; short-lived scoped HMAC session tokens with sub, plan, iat, exp, jti; jti revocation; sliding-window rate limits; kill switches; no secrets in client bundles; strict CORS allowlist configured for production.
Privacy	Complete	Pseudonymous identities; screenshot retention opt-in with deletion; data-export controls via the API; structured audit logging with request ids.
Data rights	Complete	Comparison quotes only where licensed (unlicensed sports fail closed to INSUFFICIENT MARKET DATA); closing references from providers independent of the model input feed; raw captures retained with hashes.
Responsible gambling	Complete	Persistent disclaimer banner; 21+ and 1-800-GAMBLER in the footer; no urgency language (machine-checked); no sizing, allocation, or timing guidance; copy emphasizes record-keeping.
Store compliance	Complete posture	Extension is read-only: no bet placement, preparation, or submission; no credential collection; no anti-bot bypass; opt-in per-origin permissions; remote kill switch; versioned adapters with selector-drift monitoring; NO MODEL COVERAGE for real books until a licensed possession-level feed and calibration corpus exist.
Statistical gate	Enforced in code	Preregistered parameters fixed: N=600, +1.0 pp, one-sided alpha 0.0125, 10,000 cluster-bootstrap replicates, seed 20261024, two attempts maximum, no pooling; interim descriptive only; cluster floor 30.
Compliance approval record	Default deny	Commercial launch requires a recorded gate PASS and an explicit compliance approval record per sport. Current state: NFL, NBA, UFC all denied or pending.

Table 8-1. Launch checklist. All items are enforced or documented in the build.

Appendix A. Prohibited Vocabulary Ruleset

This appendix enumerates the prohibited vocabulary for compliance review. It is the ruleset the copy linter enforces; it is deliberately not rendered as product copy — the in-app Docs surface reports only the lint result. Safe contexts are allowlisted before patterns run: "GATE PASSED", "GATE PASS", "passed", gate-verdict status tokens, and the required negated disclaimer forms ("this is not a betting recommendation").

Category	Prohibited terms and patterns
Pick categoricals	Lean Over, Lean Under, Lean(s), Pass, PASS, Avoid, Fade
Unproven advantage claims	advantage over the market (as a claim), positive-expectation abbreviations, "value" as a noun for a price judgement, lock / locked, best bet, recommend / recommendation (non-negated), guarantee(d), beat the book, "you beat the market", profitable, winning strategy, positive ROI
Sizing and exposure advice	stake(s), bankroll, hedge / hedging, cash out, exposure
Urgency	act now, before the line moves, snipe, bet now, lock it in, hurry, limited time, don't miss, chase

Table A-1. The linter ruleset with word-boundary matching and safe-context masking.

Appendix B. Reference Deployment Notes

This build is the functional reference implementation of the specification. Next.js 16 with App Router API routes stands in for the FastAPI service; SQLite through Prisma stands in for PostgreSQL (identical models); an in-process store stands in for Redis for revocation, rate limits, and kill switches; a local storage folder stands in for object storage. The domain layer — odds mathematics, the Hurdle Router, the append-only ledger with hash chains, grading with reason codes, the cluster bootstrap, and the preregistered gate — is pure, deterministic TypeScript shared verbatim with the production design.

- All simulated providers are labelled as simulated data on every surface: comparison quotes (DEMO-CMP-A), closing references (DEMO-CLOSE-1 and DEMO-CLOSE-2, namespace-disjoint from the model input feed), settlement (DEMO-SETTLE-1).
- Every calculation is reproducible from stored snapshots, model version, seed, and provenance; the Gate Monitor exposes a deterministic recompute control (fixed seed 20261024) that must match the stored attempt record.
- The vision extractor (vlm-extract-v1) reads slip fields only; charter rule 9 (no LLM in the decision path) is structural — probabilities, ratings, and gate decisions never touch a language model.
- Observability in the reference build: structured audit events with request ids, slip-upload and confirmation rates, OCR confidence distribution, ledger creation rate, closing-reference capture rate, stale-quote rate, model-coverage rate, alert delivery and suppression, entitlement failures, kill-switch state, and gate progress by sport.